

Tech Challenge – Fase 4 – Plataforma de Avaliações Serverless

Integrantes Diogo Santos RM369808 Blenda de Assunção Cardoso Gaspar RM369857 Luiz Alexandre de Barros RM369159

Repositório

<https://github.com/diogo-vds/techchallengefase4>

Vídeo de demonstração

<https://drive.google.com/file/d/1dfWiErqFTgE1cZK7InjzpwoPdGGps2Jr/view>

1. Objetivo

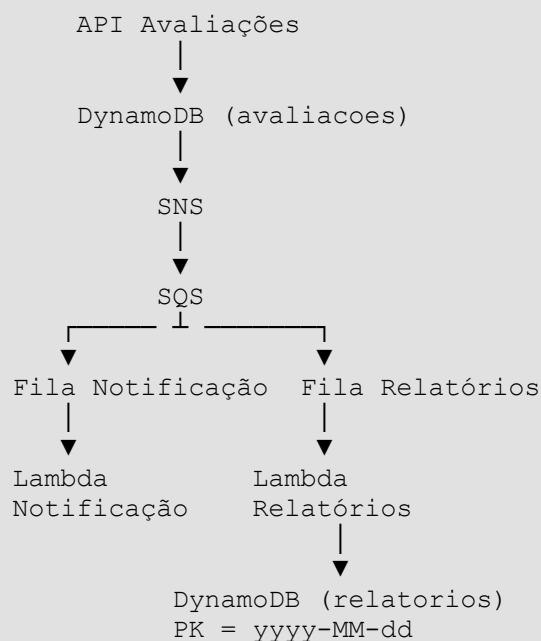
Este projeto implementa uma plataforma de coleta de feedbacks de aulas utilizando uma arquitetura baseada em serviços AWS.

A solução permite:

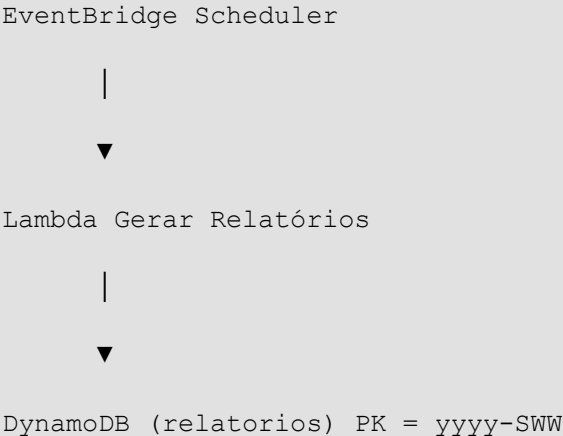
- registrar avaliações dos alunos;
- processar eventos de forma assíncrona;
- notificar administradores quando avaliações críticas são recebidas;
- gerar relatórios diários e semanais automaticamente;
- armazenar os dados em banco NoSQL utilizando DynamoDB.

2. Arquitetura da solução

Fluxo de Avaliações



Fluxo Agendado



3. Tecnologias utilizadas

Tecnologia	Utilização
Java 21	Desenvolvimento das APIs REST
Spring Boot	Framework para construção das APIs
Spring Security	Autenticação e autorização dos endpoints
Docker	Containerização das aplicações
Amazon ECS Fargate	Execução das APIs em containers
Amazon ECR	Armazenamento das imagens Docker
Amazon DynamoDB	Persistência NoSQL das avaliações e relatórios
Amazon SNS	Publicação de eventos
Amazon SQS	Filas para processamento assíncrono
AWS Lambda	Processamento Serverless
Amazon EventBridge Scheduler	Agendamento automático da geração de relatórios
Amazon CloudWatch Logs	Centralização dos logs das APIs e funções Lambda
Amazon CloudWatch Alarms	Monitoramento de erros das funções Serverless
Amazon CloudWatch Dashboard	Visualização consolidada de métricas da solução
AWS IAM	Gerenciamento de permissões e segurança
GitHub Actions	Pipeline de Integração Contínua e Deploy Contínuo (CI/CD)
Terraform	Infraestrutura como Código (IaC)

4. Componentes da solução

API Avaliações

Responsável por:

- receber avaliações dos alunos;
- validar os dados recebidos;
- verificar a criticidade da avaliação;
- persistir as avaliações no DynamoDB;
- publicar um evento no Amazon SNS.

A API é destinada aos alunos e seus endpoints são protegidos utilizando Spring Security, permitindo acesso apenas mediante autenticação.

Endpoint:

POST /avaliacoes

Exemplo:

```
{
  "descricao": "Professor explicou muito bem",
  "nota": 5
}
```

Tasks (1/1)							Last updated July 29, 2026, 11:43 (UTC-3:00)	Stop
Filter tasks by property or value							Filter desired status Any desired status	Filter launch type Any launch type
Launch type	Platform ...	Private IP	Public IP	CPU	Memory	Availabili...		
Fargate	1.4.0	172.31.91.209	34.205.156.69	1 vCPU	2 GiB	us-east-1c		

Tech Challenge2-fase 4 > avaliacoes

Save Share

POST http://34.205.156.69:8080/avaliacoes Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

Headers 8 hidden

Key	Value	Description
Authorization	Basic dXNlcjp1c2VyMTIz	
Accept	application/json	

Body 200 OK · 2.01 s · 424 B Save Response

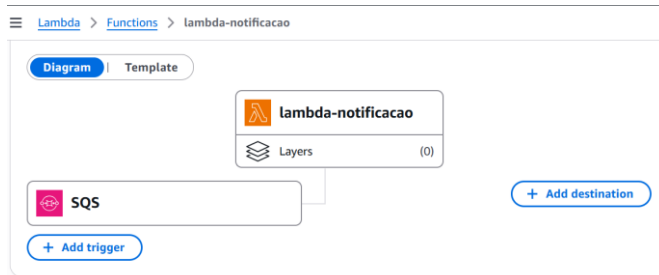
JSON Preview Visualize

```
1 {
2   "mensagem": "Avaliação salva com sucesso",
3   "id": "b9611d99-4bf6-40f8-9080-45d3467f5297"
4 }
```

Lambda Notificação

Função Serverless responsável por:

- consumir mensagens da fila de notificações;
- identificar avaliações críticas;
- simular o envio de e-mails aos administradores;
- registrar toda execução no CloudWatch.



Lambda Relatórios

Responsável por:

- consumir mensagens da fila de relatórios;
- consolidar as avaliações recebidas;
- armazenar os dados no DynamoDB utilizando a chave do dia.

Exemplo:

PK = 2026-07-29



Lambda Gerar Relatórios

Executada automaticamente pelo EventBridge Scheduler.

Responsável por:

- consultar as avaliações da semana;
- consolidar os dados da semana;

- armazenar o relatório semanal.

Exemplo:

```
PK = 2026-S07
```

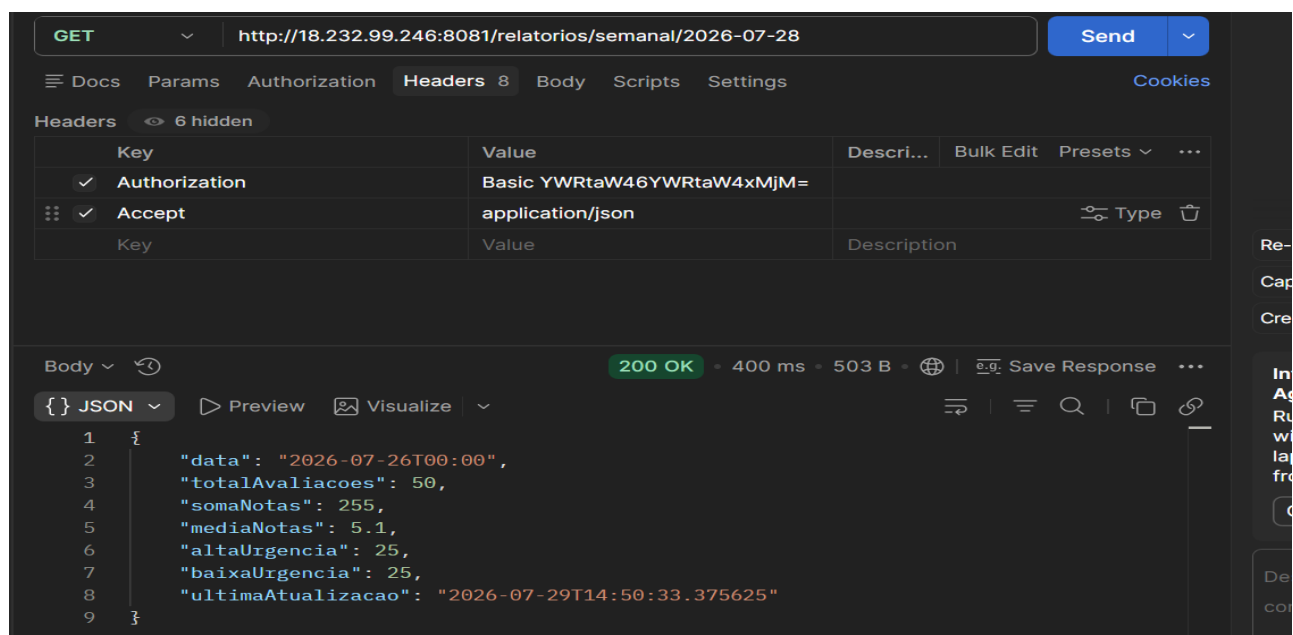
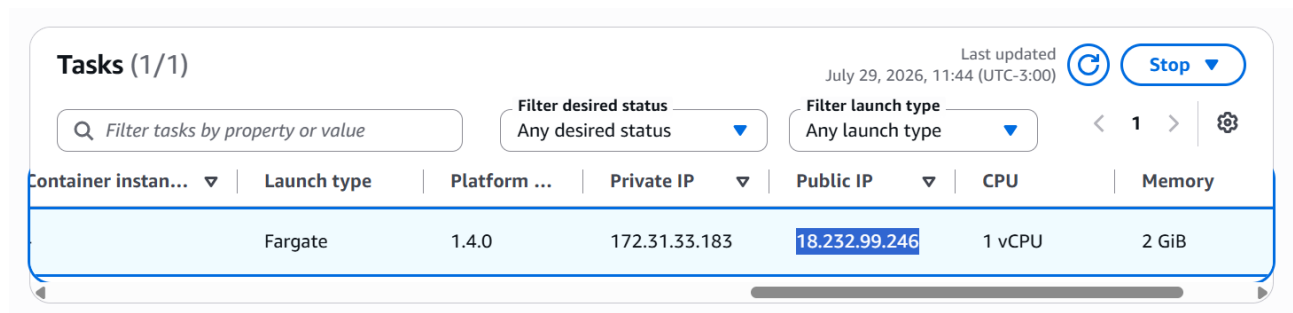


API Relatórios

A API é destinada aos administradores e seus endpoints são protegidos utilizando Spring Security, permitindo acesso apenas mediante autenticação e autorização. Disponibiliza consultas dos relatórios gerados.

Exemplo:

```
GET /relatorios/semanal/2026-07-28
```



5. Serviços AWS utilizados

Serviço	Finalidade
ECS Fargate	Hospedagem das APIs
ECR	Repositório das imagens Docker
DynamoDB	Persistência NoSQL
SNS	Publicação de eventos
SQS	Filas assíncronas
Lambda	Processamento Serverless
EventBridge Scheduler	Agendamento
CloudWatch	Logs
IAM	Controle de acesso

Infraestrutura como Código

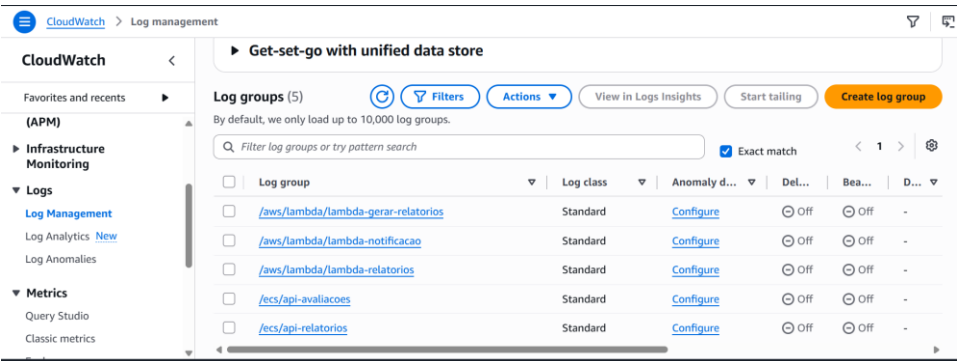
A infraestrutura da solução foi modelada utilizando Terraform, permitindo o provisionamento automatizado dos principais recursos da AWS, como ECS, ECR, DynamoDB, SNS, SQS, EventBridge e IAM. Essa abordagem garante reprodutibilidade, versionamento da infraestrutura e facilidade de manutenção.

6. Monitoramento

O monitoramento da solução é realizado através do Amazon CloudWatch.

São registrados:

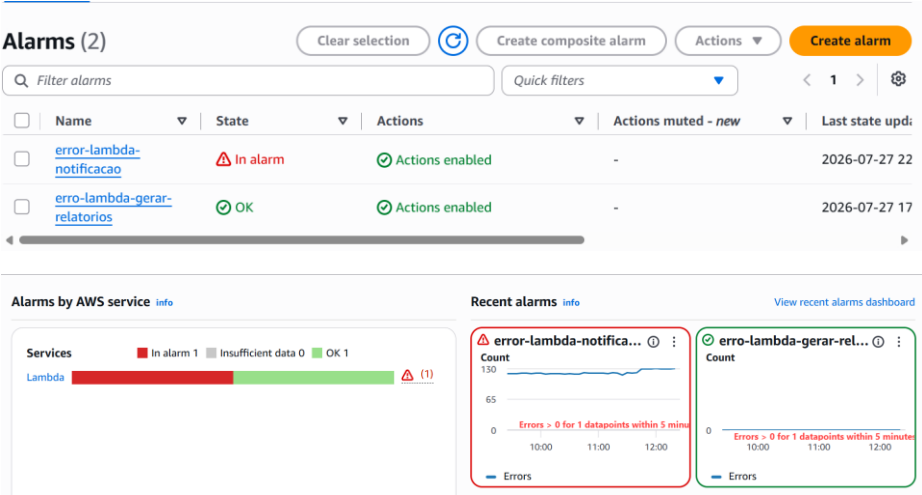
logs das APIs Spring Boot; logs das funções Lambda; execuções agendadas do EventBridge Scheduler; processamento das filas SQS; erros durante a execução das funções Serverless.



Além dos logs, foram configurados CloudWatch Alarms para monitoramento de falhas nas funções Lambda:

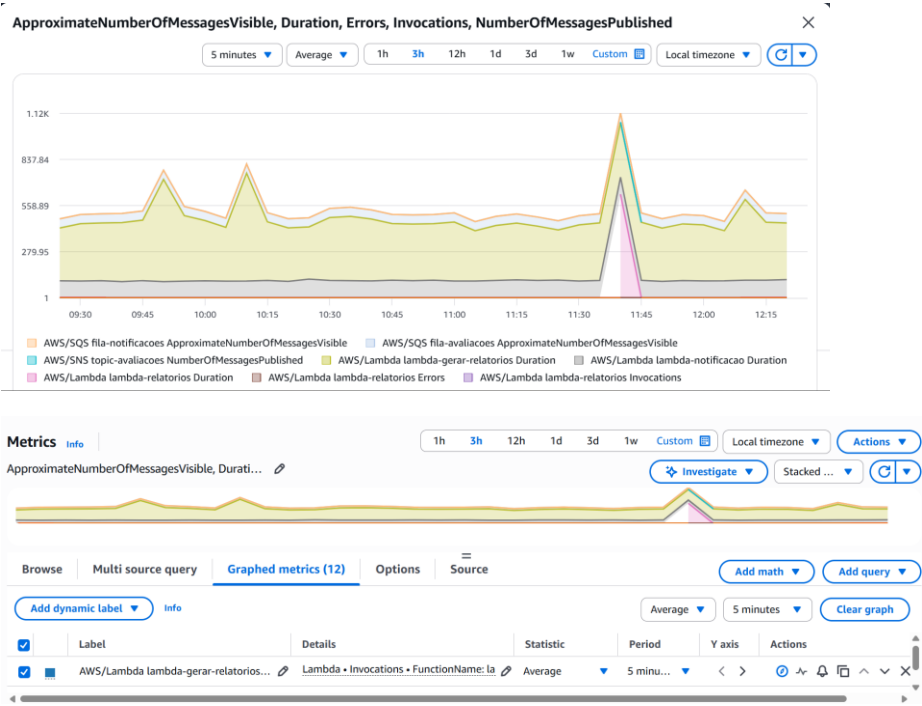
Alarme	Finalidade
error-lambda-notificacao	Monitora erros de execução da função

Alarme	Finalidade
	responsável pelas notificações.
error-lambda-gerar-relatorios	Monitora erros de execução da função responsável pela geração automática dos relatórios.



Esses alarmes permitem identificar rapidamente falhas no processamento assíncrono e facilitam a atuação corretiva durante a operação da solução.

Além dos alarmes, foi criado um CloudWatch Dashboard para centralizar a visualização das principais métricas da solução, permitindo acompanhar a saúde das APIs, funções Lambda e filas SQS em um único painel.



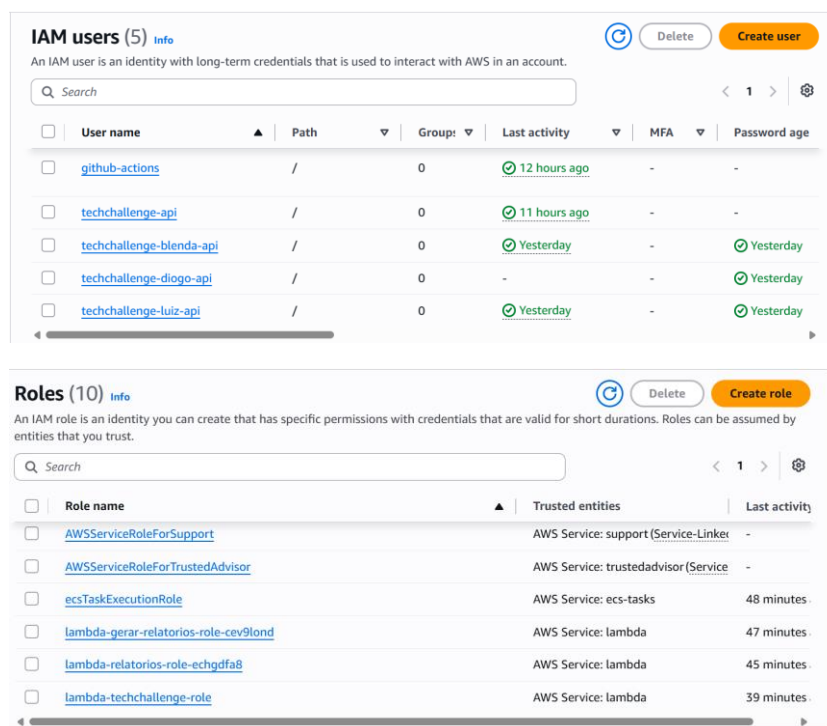
7. Segurança

A solução utiliza o modelo de segurança baseado em IAM e Spring Security, garantindo proteção tanto na infraestrutura quanto nas APIs.

Segurança da infraestrutura

Foram configuradas permissões específicas utilizando IAM para:

ECS acessar imagens armazenadas no Amazon ECR; APIs publicarem mensagens no Amazon SNS; funções Lambda consumirem mensagens das filas SQS; funções Lambda acessarem as tabelas do DynamoDB; EventBridge Scheduler executar a Lambda responsável pela geração dos relatórios.



Segurança das APIs

As APIs REST utilizam Spring Security para autenticação e autorização dos usuários.

```
@GetMapping("/{semana}/{data}") @Dev
@PreAuthorize("hasAnyRole('ADMIN')")
@Operation(summary = "Buscar relatório por semana",
description = "Retorna um relatório específico baseado na semana fornecida")
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Relatório encontrado com sucesso"),
    @ApiResponse(responseCode = "401", description = "Não autorizado - Credenciais inválidas"),
    @ApiResponse(responseCode = "403", description = "Acesso negado - Permissão insuficiente"),
    @ApiResponse(responseCode = "404", description = "Relatório não encontrado"),
    @ApiResponse(responseCode = "400", description = "ID inválido")
})
public ResponseEntity<RelatorioDiarioResponse> buscarPorSemana(
    @Parameter(description = "Data do relatório", required = true, example = "2024-06-01")
    @PathVariable String data) {
```

Foram definidos perfis distintos de acesso:

Alunos: possuem acesso à API de Avaliações, responsável pelo envio dos feedbacks. Administradores: possuem acesso à API de Relatórios, responsável pela consulta dos relatórios consolidados.

Essa separação garante que apenas usuários autorizados possam acessar informações

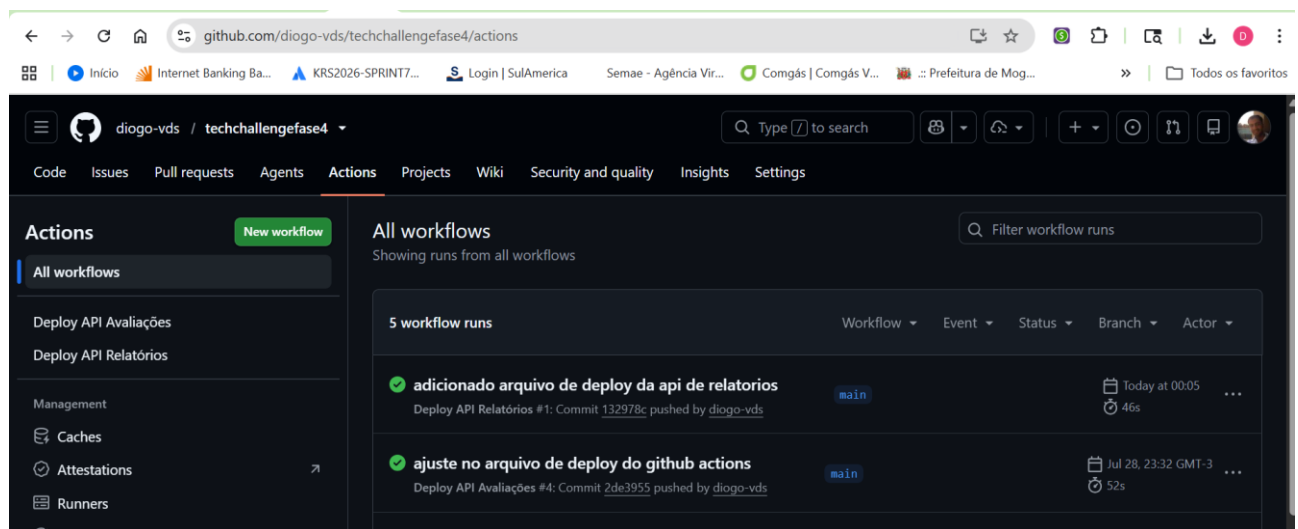
administrativas da plataforma.

8. Pipeline CI/CD

Cada API possui um workflow independente no GitHub Actions.

Fluxo do deploy:

```
Push na branch main
↓
GitHub Actions
↓
Build Maven
↓
Build Docker
↓
Push para Amazon ECR
↓
Atualização automática do ECS Service
```



9. Como executar

Pré-requisitos

- Java 21
- Maven

- Docker
- AWS CLI
- Terraform

Execução local api-avaliacoes

```
mvn clean package
```

```
java -jar avaliacao-0.0.1-SNAPSHOT.jar
```

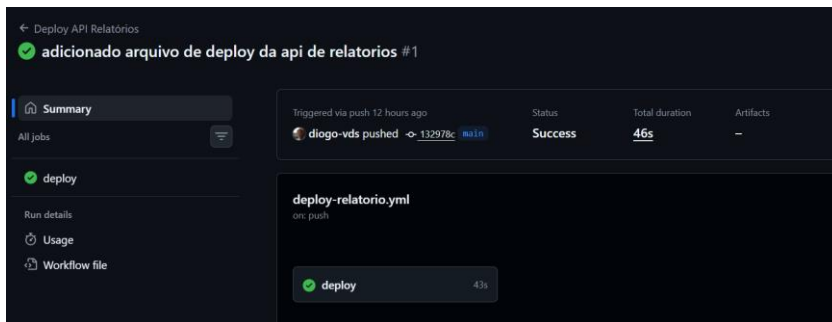
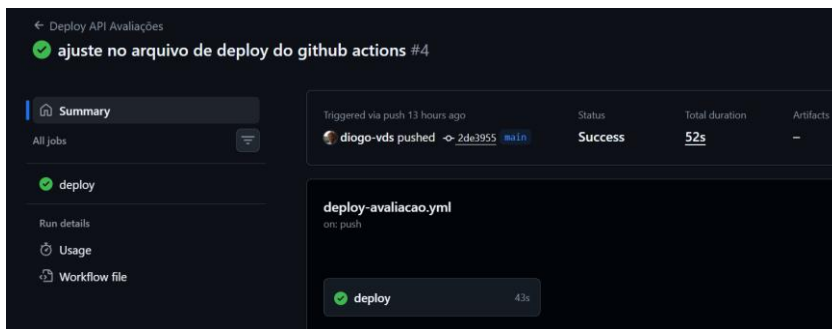
Execução local api-relatorios

```
mvn clean package
```

```
java -jar relatorios-service-1.0.0.jar
```

10. Deploy

O deploy é realizado automaticamente através do GitHub Actions.



A infraestrutura da aplicação pode ser provisionada utilizando Terraform.

Conclusão

A solução implementa uma arquitetura baseada em microsserviços e processamento assíncrono utilizando serviços gerenciados da AWS, explorando conceitos de computação em nuvem, Serverless, mensageria, infraestrutura como código e integração contínua. O uso combinado de ECS Fargate, AWS Lambda, Amazon SNS, Amazon SQS, Amazon DynamoDB e Amazon EventBridge permite uma aplicação escalável, desacoplada, resiliente e de fácil manutenção.